Reinforcement Learning

Dr Stephen James

Autumn Term 2026

Imperial College London

Reinforcement Learning
Lecture 3: Model-free Monte Carlo Methods

Dr Stephen James

Autumn Term 2026

Imperial College London

Outline

1. Introduction

2. Monte Carlo Policy Evaluation

3. Monte Carlo Control

3.1 On-policy

3.2 Off-policy

Introduction

1. Introduction

2. Monte Carlo Policy Evaluation

3. Monte Carlo Control

3.1 On-policy

3.2 Off-policy

From perfect knowledge to learning from experience

Week 2: Perfect world - we know everything

⬇️

Week 3: Real world - we learn through trial and error

Today's Journey

- From Dynamic Programming to Monte Carlo Methods
- Learn value functions by averaging sample returns
- Control without knowing the transition model
- Handle the exploration vs exploitation dilemma

What's the fundamental limitation of Dynamic Programming?

DP Requires Perfect Knowledge

- **Transition dynamics**: $P(s'|s, a)$ for all s, a, s'
- **Reward function**: $R(s, a, s')$ for all transitions
- **Complete state space**: Know all possible states

Reality Check:

- **Robotics**: Don't know exact physics, sensor noise, wear and tear
- **Games**: Opponent behavior unknown and changing
- **Finance**: Market dynamics are stochastic and non-stationary
- **Healthcare**: Patient responses are highly individual

We need methods that learn from experience, not perfect models!

What if we could learn directly from experience?

Definition (Monte Carlo Methods)
Methods that learn value functions and policies from sample episodes of experience, without requiring a model of the environment.

Key Insight:

$$V^{\pi}(s) = \mathbb{E}[G_t | S_t = s, \pi]$$

If we can't compute this expectation analytically, we can *estimate* it by averaging!

Monte Carlo Approach:

1. Generate episodes following policy π
2. For each visit to state s, observe the actual return G
3. Average these returns: $V^{\pi}(s) \approx \frac{1}{N} \sum_{i=1}^{N} G_i$

Law of Large Numbers guarantees convergence!

What if we could learn directly from experience?

Definition (Monte Carlo Methods)
Methods that learn value functions and policies from sample episodes of experience, without requiring a model of the environment.

Key Insight:

$$V^{\pi}(s) = \mathbb{E}[G_t | S_t = s, \pi]$$

If we can't compute this expectation analytically, we can *estimate* it by averaging!

Monte Carlo Approach:

1. Generate episodes following policy π
2. For each visit to state s, observe the actual return G
3. Average these returns: $V^{\pi}(s) \approx \frac{1}{N} \sum_{i=1}^{N} G_i$

Law of Large Numbers guarantees convergence!

What if we could learn directly from experience?

Definition (Monte Carlo Methods)
Methods that learn value functions and policies from sample episodes of experience, without requiring a model of the environment.

Key Insight:

$$V^{\pi}(s) = \mathbb{E}[G_t | S_t = s, \pi]$$

If we can't compute this expectation analytically, we can *estimate* it by averaging!

Monte Carlo Approach:

1. Generate episodes following policy π
2. For each visit to state s, observe the actual return G
3. Average these returns: $V^{\pi}(s) \approx \frac{1}{N} \sum_{i=1}^{N} G_i$

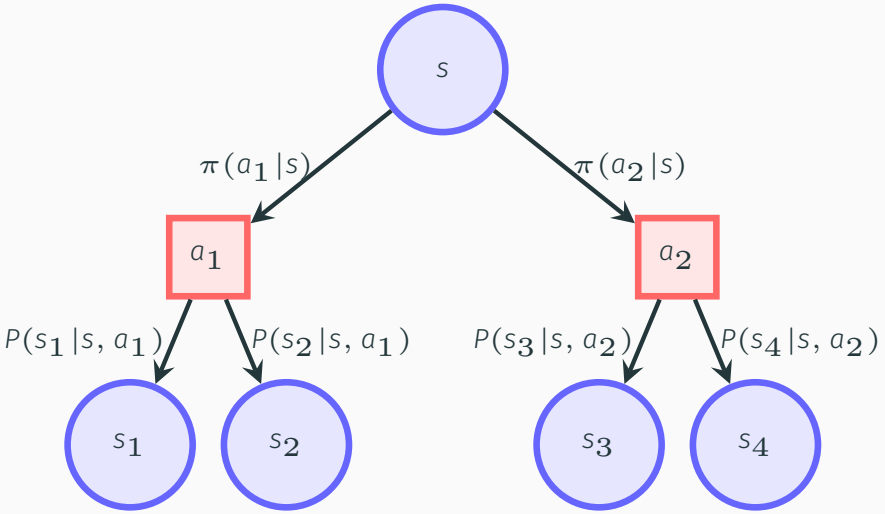
Law of Large Numbers guarantees convergence!

How do Monte Carlo and Dynamic Programming differ?

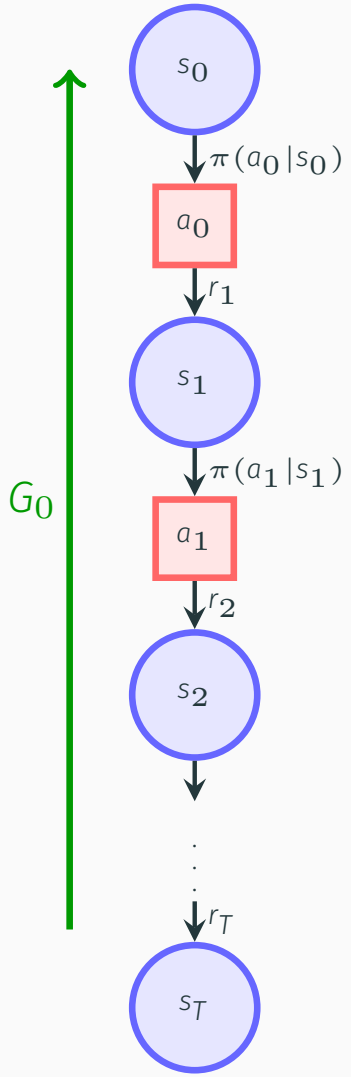
Aspect	Dynamic Programming	Monte Carlo
Requirements	Complete model	Only sample episodes
Updates	Based on Bellman equation	Based on actual returns
Bootstrapping	Uses estimates of other states	No bootstrapping
Convergence	Guaranteed with exact computation	Guaranteed by Law of Large Numbers
Computation	Sweep through all states	Focus on visited states only

What does this look like in terms of backup diagrams?

DP: One-step lookahead



MC: Full episode trajectory



Monte Carlo Policy Evaluation

1. Introduction

2. Monte Carlo Policy Evaluation

3. Monte Carlo Control

3.1 On-policy

3.2 Off-policy

How do we actually estimate value functions using Monte Carlo?

Recall the definition:

$$V^{\pi}(s) = \mathbb{E}[G_t | S_t = s, \pi]$$

Monte Carlo Estimation:

1. Generate episodes following policy π
2. For each episode, compute the return G_t from each state s
3. Average all returns observed from state s:

$$V^{\pi}(s) \approx \frac{1}{n} \sum_{i=1}^{n} G_i(s)$$

Key insight: As we collect more episodes, our estimates converge to the true values by the Law of Large Numbers.

But there's a complication...

What happens if a state is visited multiple times in one episode?

The Issue: A state might be visited multiple times in one episode

Example Episode: $s_1 \to s_2 \to s_1 \to s_3$

Two Approaches

- **First-visit**: Only use first visit to s_1 (with return from that point)
- **Every-visit**: Use both visits to s_1 (with their respective returns)

Both approaches converge to $V^\pi(s)$, but they can have different convergence rates and variance properties.

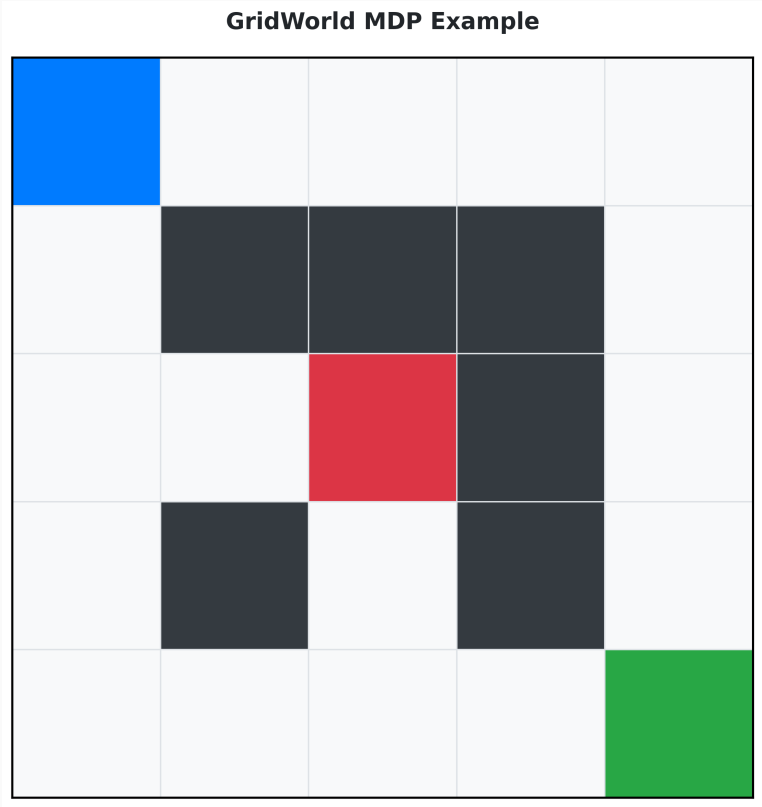
How do we implement Monte Carlo policy evaluation?

First-visit MC Policy Evaluation for V^π

Input: Policy π to be evaluated
Initialize:
$V(s) \leftarrow$ arbitrary for all $s \in \mathcal{S}$
Returns$(s) \leftarrow$ empty list for all $s \in \mathcal{S}$

repeat
    Generate episode following π: $S_0, A_0, R_1, S_1, A_1, R_2, \ldots, S_{T-1}, A_{T-1}, R_T$
    $G \leftarrow 0$
    for each step $t = T-1, T-2, \ldots, 0$ **do**
        $G \leftarrow R_{t+1} + \gamma G$
        if S_t not in $S_0, S_1, \ldots, S_{t-1}$ **then**
            Append G to Returns(S_t)
            $V(S_t) \leftarrow$ average(Returns(S_t))
        end if
    end for
until converged

Example: Monte Carlo policy evaluation in action on GridWorld



Same 5×5 GridWorld as in the DP lecture

MDP Setup:

- $\mathcal{A}$: {Up, Down, Left, Right}
- R: Goal=+1, Trap=-1, Step=0.0
- $\gamma = 0.9$

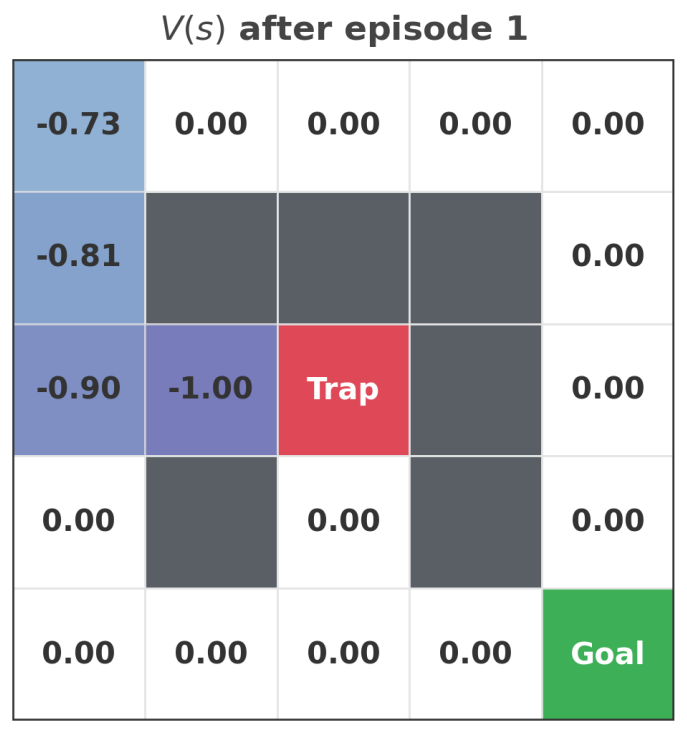
Key difference to DP:

- We don't know $P(s', r|s, a)$
- We can only run episodes and observe what happens

Policy to Evaluate: Random policy
Initialise: $V(s) = 0$ for all s

GridWorld Episode 1: The agent falls into the trap



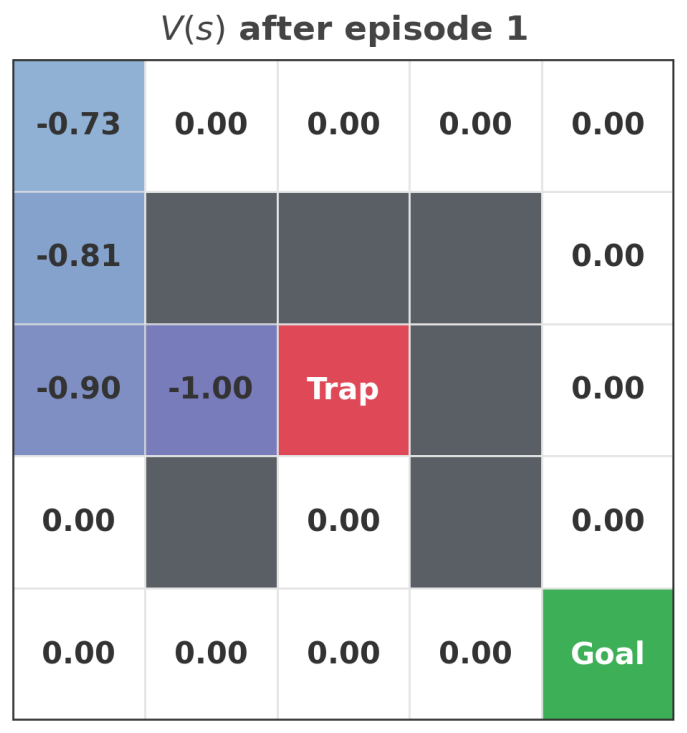


Returns (working backwards): $G = R_{t+1} + \gamma G$, so $G = -1,\ -0.9,\ -0.81,\ -0.729$ for the 4 visited states.

Only visited states are updated. Every state on the path now believes it is bad, even $(0, 0)$.

GridWorld Episode 1: The agent falls into the trap



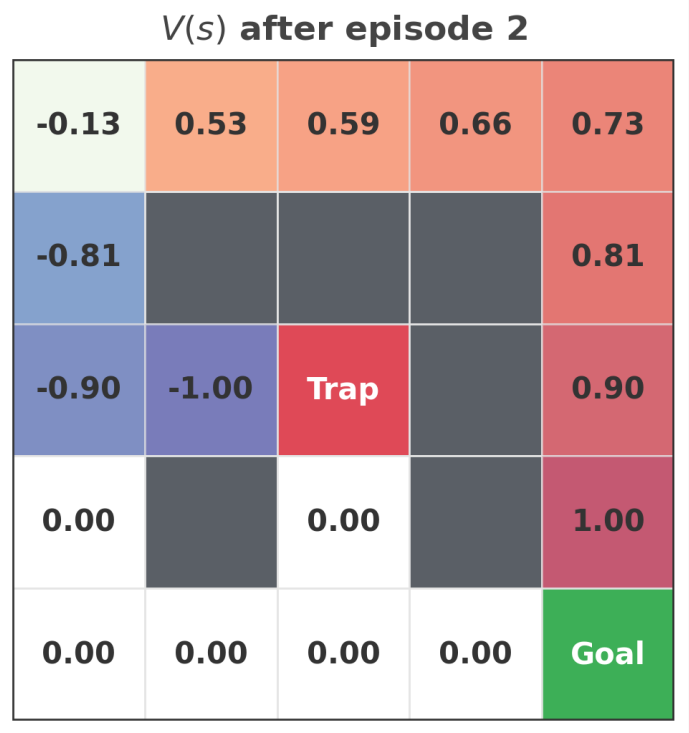


Returns (working backwards): $G = R_{t+1} + \gamma G$, so $G = -1,\ -0.9,\ -0.81,\ -0.729$ for the 4 visited states.

Only visited states are updated. Every state on the path now believes it is bad, even $(0, 0)$.

GridWorld Episode 2: The agent reaches the goal



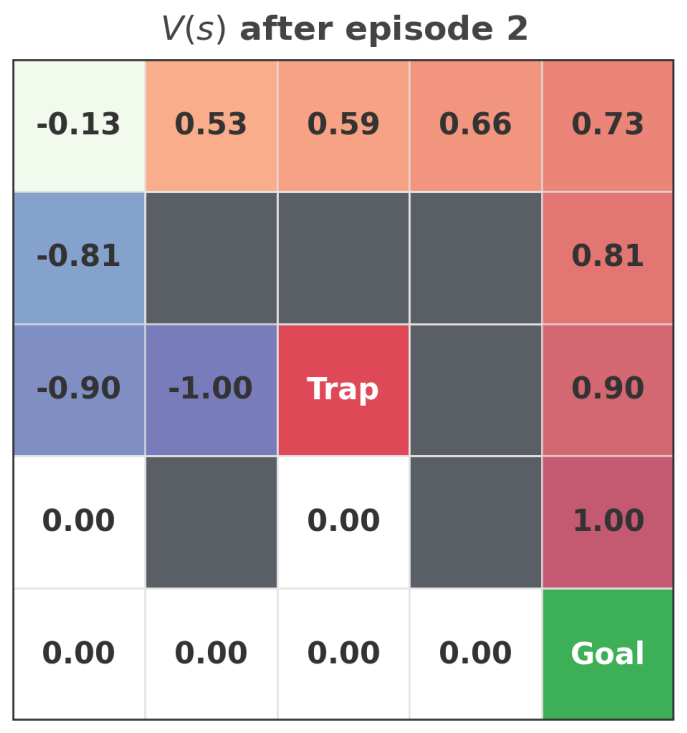


Returns (working backwards): $+1,\ 0.9,\ 0.81,\ \ldots,\ 0.478$ for the 8 visited states.

$V(0,0)$ is the average of both episodes: $\frac{1}{2}(-0.729 + 0.478) \approx -0.13$.

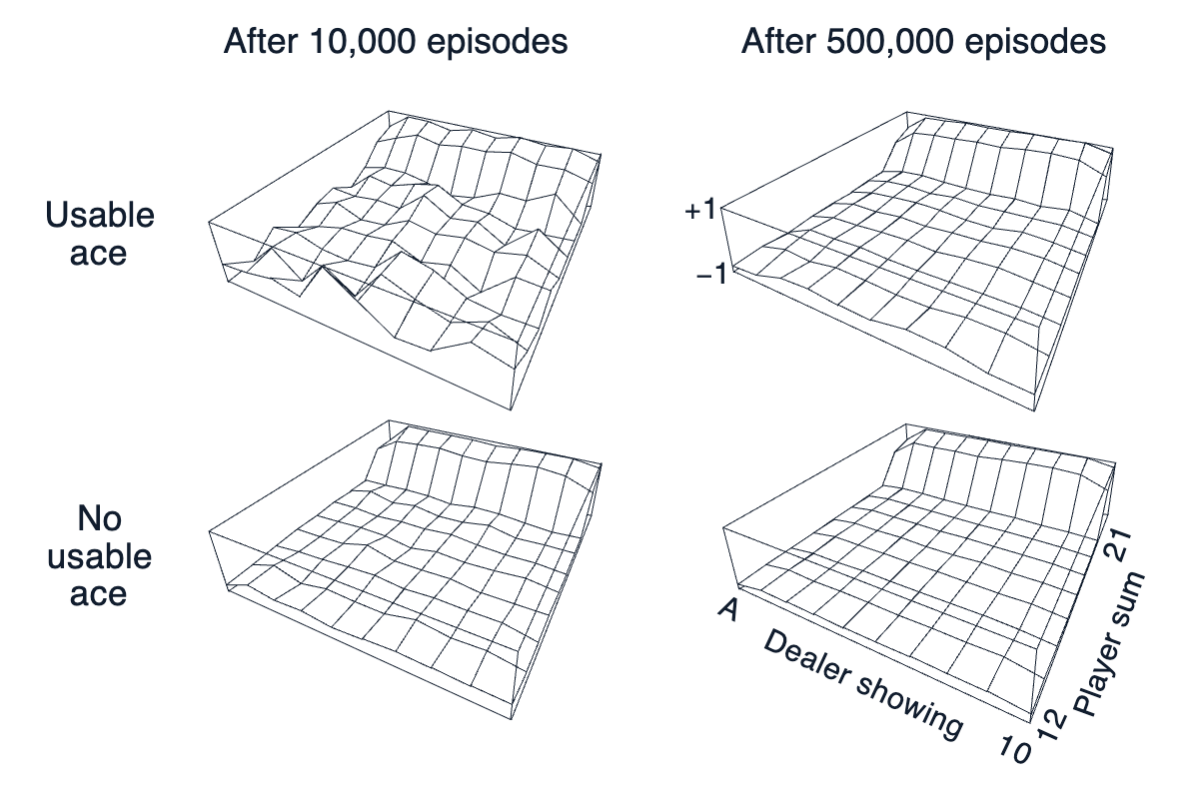
GridWorld Episode 2: The agent reaches the goal





Returns (working backwards): $+1,\ 0.9,\ 0.81,\ \ldots,\ 0.478$ for the 8 visited states.
$V(0,0)$ is the average of both episodes: $\frac{1}{2}(-0.729 + 0.478) \approx -0.13$.

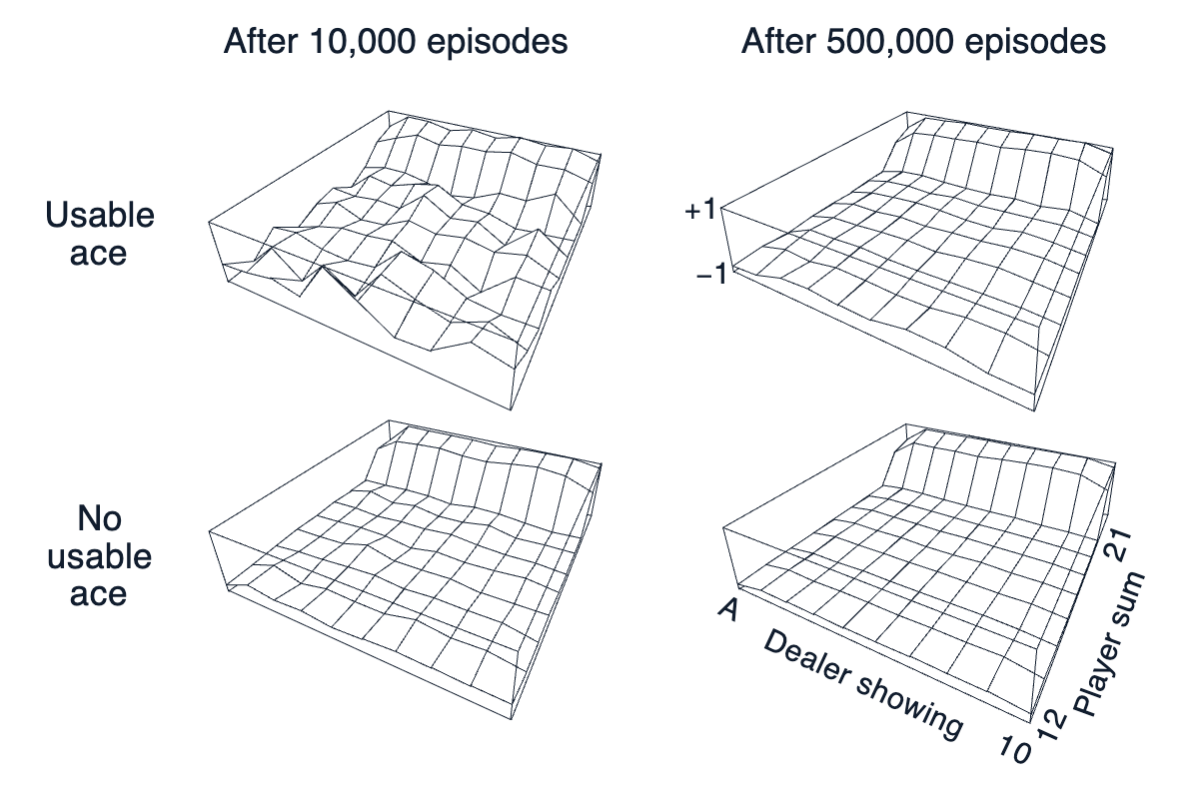
Example: MC policy evaluation on a more complex case, Blackjack



Blackjack as an MDP:
States: (player sum, dealer showing, usable ace?)
Actions: Hit or Stick
Rewards: +1 (win), -1 (lose), 0 (draw)
Usable Ace: An ace that can be counted as 11 without going over 21
Policy to Evaluate: Stick if sum $\geq$ 20, else Hit

Why Monte Carlo Works Well Here:
Model-free: Don't need to know card probabilities
Episodic: Games have clear endings
Simple: Just play games and average results!

Example: MC policy evaluation on a more complex case, Blackjack



Blackjack as an MDP:
States: (player sum, dealer showing, usable ace?)
Actions: Hit or Stick
Rewards: +1 (win), -1 (lose), 0 (draw)
Usable Ace: An ace that can be counted as 11 without going over 21
Policy to Evaluate: Stick if sum $\geq$ 20, else Hit

Why Monte Carlo Works Well Here:
Model-free: Don't need to know card probabilities
Episodic: Games have clear endings
Simple: Just play games and average results!

🎮 Mentimeter Poll

menti.com - Code: **2987 9345**

What's the main advantage of Monte Carlo over Dynamic Programming?

- A) Faster convergence
- B) Uses less memory
- C) Better theoretical guarantees
- D) No model required
- E) More accurate results

menti.com - Code: **2987 9345**

Why must Monte Carlo methods wait until the end of an episode to update values?

- A) Because the Bellman equation requires complete trajectories
- B) Because we need the actual return G_t, which depends on all future rewards
- C) Because of the Markov property
- D) Because we don't have a model
- E) Because the discount factor requires it

🎮 Mentimeter Poll: All clear so far?



menti.com — Code: **2987 9345**

Clarity on Monte Carlo Policy Evaluation?

- A) All clear
- B) Why MC doesn’t need a model unclear
- C) First-visit vs every-visit unclear
- D) How returns are computed unclear
- E) Difference from DP unclear

Monte Carlo Control

1. Introduction

2. Monte Carlo Policy Evaluation

3. Monte Carlo Control

3.1 On-policy

3.2 Off-policy

We can evaluate policies and learn V^π, but can we extract a policy from V^π?

What we've shown so far:

- We can estimate $V^\pi(s)$ from sample episodes for a given policy π

The Control Problem: Without a model, state values alone aren't sufficient!

With a model:

$$\pi^*(s) = \text{argmax}_a \sum_{s',r} P(s', r|s, a)[r + \gamma V^\pi(s')]$$

Without a model: We need to know $Q^\pi(s, a)$ directly!

$$\pi^*(s) = \text{argmax}_a Q^\pi(s, a)$$

So we must estimate action values, not just state values

We can evaluate policies and learn V^π, but can we extract a policy from V^π?

What we've shown so far:

- We can estimate $V^\pi(s)$ from sample episodes for a given policy π

The Control Problem: Without a model, state values alone aren't sufficient!

With a model:

$$\pi^*(s) = \text{argmax}_a \sum_{s',r} P(s', r|s, a)[r + \gamma V^\pi(s')]$$

Without a model: We need to know $Q^\pi(s, a)$ directly!

$$\pi^*(s) = \text{argmax}_a Q^\pi(s, a)$$

So we must estimate action values, not just state values

We can evaluate policies and learn V^π, but can we extract a policy from V^π?

What we've shown so far:

- We can estimate $V^\pi(s)$ from sample episodes for a given policy π

The Control Problem: Without a model, state values alone aren't sufficient!

With a model:

$$\pi^*(s) = \text{argmax}_a \sum_{s',r} P(s', r|s, a)[r + \gamma V^\pi(s')]$$

Without a model: We need to know $Q^\pi(s, a)$ directly!

$$\pi^*(s) = \text{argmax}_a Q^\pi(s, a)$$

So we must estimate action values, not just state values

menti.com - Code: **2987 9345**

Why do we need to estimate $Q(s, a)$ rather than just $V(s)$ for model-free control?

- A) Because $Q(s, a)$ is more accurate than $V(s)$
- B) Because $Q(s, a)$ converges faster
- C) Because we cannot do policy improvement without knowing transition probabilities
- D) Because the Bellman equation requires it
- E) Because $V(s)$ doesn't exist without a model

How do we estimate action values?

State-Action Pairs:

- A **visit** to state-action pair (s, a): state s is visited and action a is taken
- **First-visit MC**: Average returns following first visits to (s, a)
- **Every-visit MC**: Average returns following all visits to (s, a)

The Exploration Problem:

- Deterministic policy $\Rightarrow$ only one action per state gets sampled
- Need to explore ALL actions to learn $Q^{\pi}(s, a)$ for all a

This is the fundamental exploration challenge!

What are ways to solve the exploration problem?

Approach 1: Exploring Starts

- Assume episodes can start from any state-action pair (s, a)
- Every (s, a) pair has nonzero probability of being selected as start
- Guarantees all pairs visited infinitely often
- Useful in simulation, unrealistic in real environments

Monte Carlo ES (Exploring Starts)

Monte Carlo ES Algorithm

Initialize:
$\pi(s) \leftarrow$ arbitrary policy for all s
$Q(s, a) \leftarrow$ arbitrary values for all s, a
Returns$(s, a) \leftarrow$ empty list for all s, a
repeat
    Choose $S_0 \in \mathcal{S}, A_0 \in \mathcal{A}(S_0)$ s.t. all pairs have prob > 0
    Generate episode: $S_0, A_0, R_1, S_1, A_1, R_2, \ldots, S_{T-1}, A_{T-1}, R_T$
    $G \leftarrow 0$
    for $t = T-1, T-2, \ldots, 0$ **do**
        $G \leftarrow \gamma G + R_{t+1}$
        if (S_t, A_t) not in $(S_0, A_0), (S_1, A_1), \ldots, (S_{t-1}, A_{t-1})$ **then**
            Append G to Returns(S_t, A_t)
            $Q(S_t, A_t) \leftarrow$ average(Returns(S_t, A_t))
            $\pi(S_t) \leftarrow \text{argmax}_a Q(S_t, a)$
        end if
    end for
until converged

How can we avoid the unlikely assumption of exploring starts?

The only general way to ensure all actions are selected infinitely often is for the agent to continue to select them.

Two approaches to ensuring this:

On-policy Methods

- Evaluate and improve the policy that is used to make decisions
- The policy being learned is the same as the policy being followed
- Must use stochastic policies to maintain exploration

Off-policy Methods

- Learn about one policy while following a different policy
- **Target policy**: what we want to learn about
- **Behavior policy**: what we use to generate experience

Both require stochastic policies, but for different reasons...

How can we avoid the unlikely assumption of exploring starts?

The only general way to ensure all actions are selected infinitely often is for the agent to continue to select them.

Two approaches to ensuring this:

On-policy Methods

- Evaluate and improve the policy that is used to make decisions
- The policy being learned is the same as the policy being followed
- Must use stochastic policies to maintain exploration

Off-policy Methods

- Learn about one policy while following a different policy
- **Target policy**: what we want to learn about
- **Behavior policy**: what we use to generate experience

Both require stochastic policies, but for different reasons...

How can we avoid the unlikely assumption of exploring starts?

The only general way to ensure all actions are selected infinitely often is for the agent to continue to select them.

Two approaches to ensuring this:

On-policy Methods

- Evaluate and improve the policy that is used to make decisions
- The policy being learned is the same as the policy being followed
- Must use stochastic policies to maintain exploration

Off-policy Methods

- Learn about one policy while following a different policy
- **Target policy**: what we want to learn about
- **Behavior policy**: what we use to generate experience

Both require stochastic policies, but for different reasons...

Common stochastic policy: ϵ-greedy

ϵ-greedy policies: Most common type of stochastic policy for exploration

$$\pi(a|s) = \begin{cases} 1 - \epsilon + \frac{\epsilon}{|\mathcal{A}(s)|} & \text{if } a = \text{argmax}_{a'} Q(s, a') \\ \frac{\epsilon}{|\mathcal{A}(s)|} & \text{otherwise} \end{cases}$$

Intuition:

- With probability $(1 - \epsilon)$: Choose the greedy action
- With probability ϵ: Choose uniformly among all actions
- Parameter ϵ controls exploration vs exploitation trade-off

Examples: $\epsilon = 0$ is pure greedy (no exploration); $\epsilon = 0.1$ is mostly greedy with a small amount of exploration.

ϵ-greedy policies is an example of an **ϵ-soft policy**, meaning that $\pi(a|s) > 0$ for all s, a, but gradually shifted to a deterministic policy as $\epsilon \to 0$.

Equivalent one-line form (we will use this later):

$$\pi(a|s) = \frac{\epsilon}{|\mathcal{A}(s)|} + (1 - \epsilon)\, \mathbb{1}[a = \text{argmax}_{a'} Q(s, a')]$$

Common stochastic policy: ϵ-greedy

ϵ-greedy policies: Most common type of stochastic policy for exploration

$$\pi(a|s) = \begin{cases} 1 - \epsilon + \frac{\epsilon}{|\mathcal{A}(s)|} & \text{if } a = \text{argmax}_{a'} Q(s, a') \\ \frac{\epsilon}{|\mathcal{A}(s)|} & \text{otherwise} \end{cases}$$

Intuition:

- With probability $(1 - \epsilon)$: Choose the greedy action
- With probability ϵ: Choose uniformly among all actions
- Parameter ϵ controls exploration vs exploitation trade-off

Examples: $\epsilon = 0$ is pure greedy (no exploration); $\epsilon = 0.1$ is mostly greedy with a small amount of exploration.

ϵ-greedy policies is an example of an **ϵ-soft policy**, meaning that $\pi(a|s) > 0$ for all s, a, but gradually shifted to a deterministic policy as $\epsilon \to 0$.

Equivalent one-line form (we will use this later):

$$\pi(a|s) = \frac{\epsilon}{|\mathcal{A}(s)|} + (1 - \epsilon)\, \mathbb{1}[a = \text{argmax}_{a'} Q(s, a')]$$

Common stochastic policy: ϵ-greedy

ϵ-greedy policies: Most common type of stochastic policy for exploration

$$\pi(a|s) = \begin{cases} 1 - \epsilon + \frac{\epsilon}{|\mathcal{A}(s)|} & \text{if } a = \text{argmax}_{a'} Q(s, a') \\ \frac{\epsilon}{|\mathcal{A}(s)|} & \text{otherwise} \end{cases}$$

Intuition:

- With probability $(1 - \epsilon)$: Choose the greedy action
- With probability ϵ: Choose uniformly among all actions
- Parameter ϵ controls exploration vs exploitation trade-off

Examples: $\epsilon = 0$ is pure greedy (no exploration); $\epsilon = 0.1$ is mostly greedy with a small amount of exploration.

ϵ-greedy policies is an example of an **ϵ-soft policy**, meaning that $\pi(a|s) > 0$ for all s, a, but gradually shifted to a deterministic policy as $\epsilon \to 0$.

Equivalent one-line form (we will use this later):

$$\pi(a|s) = \frac{\epsilon}{|\mathcal{A}(s)|} + (1 - \epsilon)\, \mathbb{1}[a = \text{argmax}_{a'} Q(s, a')]$$

menti.com - Code: **2987 9345**

In an ϵ-greedy policy with $|\mathcal{A}(s)| = 4$ actions and $\epsilon = 0.2$, what is the probability of selecting the greedy action?

- A) 0.85
- B) 0.80
- C) 0.20
- D) 0.05
- E) 0.95

$$\pi(a|s) = \begin{cases} 1 - \epsilon + \frac{\epsilon}{|\mathcal{A}(s)|} & \text{if } a = \text{argmax}_{a'} Q(s, a') \\ \frac{\epsilon}{|\mathcal{A}(s)|} & \text{otherwise} \end{cases} \tag{1}$$

🎮 Mentimeter Poll: All clear so far?

menti.com — Code: **2987 9345**

Clarity on MC Control and Exploring Starts?

- A) All clear
- B) Why we need exploration unclear
- C) Exploring starts assumption unclear
- D) How MC ES differs from MC evaluation unclear
- E) What ϵ-greedy means unclear

On-policy

1. Introduction

2. Monte Carlo Policy Evaluation

3. Monte Carlo Control

3.1 On-policy

3.2 Off-policy

How do we put evaluation and improvement together for on-policy methods?

On-policy Monte Carlo Control = MC Evaluation + ϵ-greedy Improvement

Key insight: Apply GPI but maintain exploration through ϵ-greedy policies

On-policy Monte Carlo Control Algorithm

1. **Initialize**: Arbitrary ϵ-greedy policy π_0, empty $Q(s, a)$
2. **Repeat until convergence**:
   - **Policy Evaluation**: Generate episode following π_k, update $Q^{\pi_k}(s, a)$ estimates
   - **Policy Improvement**: Set π_{k+1} to be ϵ-greedy w.r.t. Q^{π_k}
3. **Converges to**: $\pi^*_{\epsilon\text{-soft}}$ (best ϵ-greedy policy)

$$\pi_0 \xrightarrow{E} Q^{\pi_0} \xrightarrow{I} \pi_1 \xrightarrow{E} Q^{\pi_1} \xrightarrow{I} \ldots \rightarrow \pi^*_{\epsilon\text{-soft}}$$

How do we put evaluation and improvement together for on-policy methods?

On-policy Monte Carlo Control = MC Evaluation + ϵ-greedy Improvement

Key insight: Apply GPI but maintain exploration through ϵ-greedy policies

On-policy Monte Carlo Control Algorithm

1. **Initialize**: Arbitrary ϵ-greedy policy π_0, empty $Q(s, a)$
2. **Repeat until convergence**:
   - **Policy Evaluation**: Generate episode following π_k, update $Q^{\pi_k}(s, a)$ estimates
   - **Policy Improvement**: Set π_{k+1} to be ϵ-greedy w.r.t. Q^{π_k}
3. **Converges to**: $\pi^*_{\epsilon\text{-soft}}$ (best ϵ-greedy policy)

$$\pi_0 \xrightarrow{E} Q^{\pi_0} \xrightarrow{I} \pi_1 \xrightarrow{E} Q^{\pi_1} \xrightarrow{I} \ldots \rightarrow \pi^*_{\epsilon\text{-soft}}$$

How do we put evaluation and improvement together for on-policy methods?

On-policy Monte Carlo Control = MC Evaluation + ϵ-greedy Improvement

Key insight: Apply GPI but maintain exploration through ϵ-greedy policies

On-policy Monte Carlo Control Algorithm

1. **Initialize**: Arbitrary ϵ-greedy policy π_0, empty $Q(s, a)$
2. **Repeat until convergence**:
   - **Policy Evaluation**: Generate episode following π_k, update $Q^{\pi_k}(s, a)$ estimates
   - **Policy Improvement**: Set π_{k+1} to be ϵ-greedy w.r.t. Q^{π_k}
3. **Converges to**: $\pi^*_{\epsilon\text{-soft}}$ (best ϵ-greedy policy)

$$\pi_0 \xrightarrow{E} Q^{\pi_0} \xrightarrow{I} \pi_1 \xrightarrow{E} Q^{\pi_1} \xrightarrow{I} \ldots \rightarrow \pi^*_{\epsilon\text{-soft}}$$

On-policy First-visit MC Control Algorithm

Algorithm parameter: small $\epsilon > 0$
Initialize:
$\pi \leftarrow$ arbitrary ϵ-soft policy
$Q(s, a) \leftarrow$ arbitrary values for all s, a
Returns$(s, a) \leftarrow$ empty list for all s, a
repeat
    Generate episode following π
    $G \leftarrow 0$
    for each step $t = T - 1, T - 2, \ldots, 0$ **do**
        $G \leftarrow \gamma G + r_{t+1}$
        if (s_t, a_t) not appears earlier **then**
            Append G to Returns(s_t, a_t)
            $Q(s_t, a_t) \leftarrow$ average(Returns(s_t, a_t))
            for all $a \in \mathcal{A}(s_t)$ **do**

$$\pi(a|s_t) \leftarrow \begin{cases} 1 - \epsilon + \frac{\epsilon}{|\mathcal{A}(s_t)|} & \text{if } a = \text{argmax}_a Q(s_t, a) \\ \frac{\epsilon}{|\mathcal{A}(s_t)|} & \text{otherwise} \end{cases}$$

            end for
        end if
    end for
until forever

Why does ϵ-greedy policy improvement work? (Proof sketch)

We want to show: If π is ϵ-soft and π' is the ϵ-greedy policy w.r.t. Q^π, then:

$$V_{\pi'}(s) \geq V_\pi(s) \quad \forall s \in \mathcal{S}$$

$$Q^\pi(s, \pi'(s)) = \sum_a \pi'(a|s) Q^\pi(s, a) \qquad \text{(definition of Q-value under new policy)}$$

$$= \frac{\epsilon}{|\mathcal{A}(s)|} \sum_a Q^\pi(s, a) + (1 - \epsilon) \max_a Q^\pi(s, a) \qquad \text{(substitute } \epsilon\text{-greedy policy)}$$

$$\geq \frac{\epsilon}{|\mathcal{A}(s)|} \sum_a Q^\pi(s, a) + (1 - \epsilon) \sum_a w(a) Q^\pi(s, a) \qquad \text{(max} \geq \text{weighted average, for any weights } w\text{)}$$

$$= \frac{\epsilon}{|\mathcal{A}(s)|} \sum_a Q^\pi(s, a) + (1 - \epsilon) \sum_a \frac{\pi(a|s) - \frac{\epsilon}{|\mathcal{A}(s)|}}{1 - \epsilon} Q^\pi(s, a)$$

(choose $w(a)$ so the π terms appear)

$$= \sum_a \frac{\epsilon}{|\mathcal{A}(s)|} Q^\pi(s, a) - \sum_a \frac{\epsilon}{|\mathcal{A}(s)|} Q^\pi(s, a) + \sum_a \pi(a|s) Q^\pi(s, a) \qquad \text{(rearrange terms)}$$

$$= \sum_a \pi(a|s) Q^\pi(s, a) = \underline{V^\pi(s)} \qquad \text{(algebra simplifies to original policy)}$$

Why does ϵ-greedy policy improvement work? (Proof sketch)

We want to show: If π is ϵ-soft and π' is the ϵ-greedy policy w.r.t. Q^π, then:

$$V_{\pi'}(s) \geq V_\pi(s) \quad \forall s \in \mathcal{S}$$

$$\begin{aligned}
Q^\pi(s, \pi'(s)) &= \sum_a \pi'(a|s) Q^\pi(s,a) && \text{(definition of Q-value under new policy)} \\
&= \frac{\epsilon}{|\mathcal{A}(s)|} \sum_a Q^\pi(s,a) + (1-\epsilon) \max_a Q^\pi(s,a) && \text{(substitute } \epsilon\text{-greedy policy)} \\
&\geq \frac{\epsilon}{|\mathcal{A}(s)|} \sum_a Q^\pi(s,a) + (1-\epsilon) \sum_a w(a) Q^\pi(s,a) && \text{(max} \geq \text{weighted average, for any weights } w) \\
&= \frac{\epsilon}{|\mathcal{A}(s)|} \sum_a Q^\pi(s,a) + (1-\epsilon) \sum_a \frac{\pi(a|s) - \frac{\epsilon}{|\mathcal{A}(s)|}}{1-\epsilon} Q^\pi(s,a) \\
& && \text{(choose } w(a) \text{ so the } \pi \text{ terms appear)} \\
&= \sum_a \frac{\epsilon}{|\mathcal{A}(s)|} Q^\pi(s,a) - \sum_a \frac{\epsilon}{|\mathcal{A}(s)|} Q^\pi(s,a) + \sum_a \pi(a|s) Q^\pi(s,a) && \text{(rearrange terms)} \\
&= \sum_a \pi(a|s) Q^\pi(s,a) = \underline{V^\pi(s)} && \text{(algebra simplifies to original policy)}
\end{aligned}$$

Key insight for step 2: The new ϵ-greedy policy can be written as: $\pi'(a|s) = \frac{\epsilon}{|\mathcal{A}(s)|} + (1-\epsilon) \cdot \mathbb{1}[a \text{ is greedy w.r.t. } Q^\pi]$

Substituting and splitting the sum:

$$\begin{aligned}
\sum_a \pi'(a|s) Q^\pi(s,a) &= \sum_a \left[\frac{\epsilon}{|\mathcal{A}(s)|} + (1-\epsilon)\, \mathbb{1}[a \text{ is greedy}] \right] Q^\pi(s,a) \\
&= \frac{\epsilon}{|\mathcal{A}(s)|} \sum_a Q^\pi(s,a) + (1-\epsilon) \underbrace{\sum_a \mathbb{1}[a \text{ is greedy}]\, Q^\pi(s,a)}_{= \max_a Q^\pi(s,a)}
\end{aligned}$$

Why does ϵ-greedy policy improvement work? (Proof sketch)

We want to show: If π is ϵ-soft and π' is the ϵ-greedy policy w.r.t. Q^π, then:

$$V_{\pi'}(s) \geq V_\pi(s) \quad \forall s \in \mathcal{S}$$

$$\begin{aligned}
Q^\pi(s, \pi'(s)) &= \sum_a \pi'(a|s) Q^\pi(s,a) && \text{(definition of Q-value under new policy)} \\
&= \frac{\epsilon}{|\mathcal{A}(s)|} \sum_a Q^\pi(s,a) + (1-\epsilon) \max_a Q^\pi(s,a) && \text{(substitute } \epsilon\text{-greedy policy)} \\
&\geq \frac{\epsilon}{|\mathcal{A}(s)|} \sum_a Q^\pi(s,a) + (1-\epsilon) \sum_a w(a) Q^\pi(s,a) && \text{(max} \geq \text{weighted average, for any weights } w\text{)} \\
&= \frac{\epsilon}{|\mathcal{A}(s)|} \sum_a Q^\pi(s,a) + (1-\epsilon) \sum_a \frac{\pi(a|s) - \frac{\epsilon}{|\mathcal{A}(s)|}}{1-\epsilon} Q^\pi(s,a) \\
& && \text{(choose } w(a) \text{ so the } \pi \text{ terms appear)} \\
&= \sum_a \frac{\epsilon}{|\mathcal{A}(s)|} Q^\pi(s,a) - \sum_a \frac{\epsilon}{|\mathcal{A}(s)|} Q^\pi(s,a) + \sum_a \pi(a|s) Q^\pi(s,a) && \text{(rearrange terms)} \\
&= \sum_a \pi(a|s) Q^\pi(s,a) = \underline{V^\pi(s)} && \text{(algebra simplifies to original policy)}
\end{aligned}$$

Key insight for step 3: For <u>any</u> weights $w(a) \geq 0$ with $\sum_a w(a) = 1$, a weighted average can never beat the best value.

Why does ϵ-greedy policy improvement work? (Proof sketch)

We want to show: If π is ϵ-soft and π' is the ϵ-greedy policy w.r.t. Q^π, then:

$$V_{\pi'}(s) \geq V_\pi(s) \quad \forall s \in \mathcal{S}$$

$$\begin{aligned}
Q^\pi(s, \pi'(s)) &= \sum_a \pi'(a|s) Q^\pi(s,a) && \text{(definition of Q-value under new policy)} \\
&= \frac{\epsilon}{|\mathcal{A}(s)|} \sum_a Q^\pi(s,a) + (1-\epsilon) \max_a Q^\pi(s,a) && \text{(substitute } \epsilon\text{-greedy policy)} \\
&\geq \frac{\epsilon}{|\mathcal{A}(s)|} \sum_a Q^\pi(s,a) + (1-\epsilon) \sum_a w(a) Q^\pi(s,a) && \text{(max} \geq \text{weighted average, for any weights } w) \\
&= \frac{\epsilon}{|\mathcal{A}(s)|} \sum_a Q^\pi(s,a) + (1-\epsilon) \sum_a \frac{\pi(a|s) - \frac{\epsilon}{|\mathcal{A}(s)|}}{1-\epsilon} Q^\pi(s,a) \\
& && \text{(choose } w(a) \text{ so the } \pi \text{ terms appear)} \\
&= \sum_a \frac{\epsilon}{|\mathcal{A}(s)|} Q^\pi(s,a) - \sum_a \frac{\epsilon}{|\mathcal{A}(s)|} Q^\pi(s,a) + \sum_a \pi(a|s) Q^\pi(s,a) && \text{(rearrange terms)} \\
&= \sum_a \pi(a|s) Q^\pi(s,a) = V^\pi(s) && \text{(algebra simplifies to original policy)}
\end{aligned}$$

Key insight for step 4: Any valid w works, so we are free to set the weights to

$$w(a) = \frac{\pi(a|s) - \frac{\epsilon}{|\mathcal{A}(s)|}}{1-\epsilon}$$

Why it is valid: π is ϵ-soft, so $\pi(a|s) \geq \frac{\epsilon}{|\mathcal{A}(s)|}$ and $w(a) \geq 0$. The weights also sum to $\frac{1-\epsilon}{1-\epsilon} = 1$.

Why this particular w? It will become clear in the next steps.

Why does ϵ-greedy policy improvement work? (Proof sketch)

We want to show: If π is ϵ-soft and π' is the ϵ-greedy policy w.r.t. Q^π, then:

$$V_{\pi'}(s) \geq V_\pi(s) \quad \forall s \in \mathcal{S}$$

$$
\begin{aligned}
Q^\pi(s, \pi'(s)) &= \sum_a \pi'(a|s) Q^\pi(s,a) && \text{(definition of Q-value under new policy)} \\
&= \frac{\epsilon}{|\mathcal{A}(s)|} \sum_a Q^\pi(s,a) + (1-\epsilon) \max_a Q^\pi(s,a) && \text{(substitute } \epsilon\text{-greedy policy)} \\
&\geq \frac{\epsilon}{|\mathcal{A}(s)|} \sum_a Q^\pi(s,a) + (1-\epsilon) \sum_a w(a) Q^\pi(s,a) && \text{(max} \geq \text{weighted average, for any weights } w) \\
&= \frac{\epsilon}{|\mathcal{A}(s)|} \sum_a Q^\pi(s,a) + (1-\epsilon) \sum_a \frac{\pi(a|s) - \frac{\epsilon}{|\mathcal{A}(s)|}}{1-\epsilon} Q^\pi(s,a) \\
& && \text{(choose } w(a) \text{ so the } \pi \text{ terms appear)} \\
&= \sum_a \frac{\epsilon}{|\mathcal{A}(s)|} Q^\pi(s,a) - \sum_a \frac{\epsilon}{|\mathcal{A}(s)|} Q^\pi(s,a) + \sum_a \pi(a|s) Q^\pi(s,a) && \text{(rearrange terms)} \\
&= \sum_a \pi(a|s) Q^\pi(s,a) = \underline{V^\pi(s)} && \text{(algebra simplifies to original policy)}
\end{aligned}
$$

Why does ϵ-greedy policy improvement work? (Proof sketch)

We want to show: If π is ϵ-soft and π' is the ϵ-greedy policy w.r.t. Q^π, then:

$$V_{\pi'}(s) \geq V_\pi(s) \quad \forall s \in \mathcal{S}$$

$$
\begin{aligned}
Q^\pi(s, \pi'(s)) &= \sum_a \pi'(a|s) Q^\pi(s,a) && \text{(definition of Q-value under new policy)} \\
&= \frac{\epsilon}{|\mathcal{A}(s)|} \sum_a Q^\pi(s,a) + (1-\epsilon) \max_a Q^\pi(s,a) && \text{(substitute } \epsilon\text{-greedy policy)} \\
&\geq \frac{\epsilon}{|\mathcal{A}(s)|} \sum_a Q^\pi(s,a) + (1-\epsilon) \sum_a w(a) Q^\pi(s,a) && \text{(max} \geq \text{weighted average, for any weights } w) \\
&= \frac{\epsilon}{|\mathcal{A}(s)|} \sum_a Q^\pi(s,a) + (1-\epsilon) \sum_a \frac{\pi(a|s) - \frac{\epsilon}{|\mathcal{A}(s)|}}{1-\epsilon} Q^\pi(s,a) \\
& && \text{(choose } w(a) \text{ so the } \pi \text{ terms appear)} \\
&= \sum_a \frac{\epsilon}{|\mathcal{A}(s)|} Q^\pi(s,a) - \sum_a \frac{\epsilon}{|\mathcal{A}(s)|} Q^\pi(s,a) + \sum_a \pi(a|s) Q^\pi(s,a) && \text{(rearrange terms)} \\
&= \sum_a \pi(a|s) Q^\pi(s,a) = \underline{V^\pi(s)} && \text{(algebra simplifies to original policy)}
\end{aligned}
$$

Why does ϵ-greedy policy improvement work? (Proof sketch)

We want to show: If π is ϵ-soft and π' is the ϵ-greedy policy w.r.t. Q^π, then:

$$V_{\pi'}(s) \geq V_\pi(s) \quad \forall s \in \mathcal{S}$$

$$\begin{aligned}
Q^\pi(s, \pi'(s)) &= \sum_a \pi'(a|s) Q^\pi(s,a) && \text{(definition of Q-value under new policy)} \\
&= \frac{\epsilon}{|\mathcal{A}(s)|} \sum_a Q^\pi(s,a) + (1-\epsilon) \max_a Q^\pi(s,a) && \text{(substitute } \epsilon\text{-greedy policy)} \\
&\geq \frac{\epsilon}{|\mathcal{A}(s)|} \sum_a Q^\pi(s,a) + (1-\epsilon) \sum_a w(a) Q^\pi(s,a) && \text{(max} \geq \text{weighted average, for any weights } w) \\
&= \frac{\epsilon}{|\mathcal{A}(s)|} \sum_a Q^\pi(s,a) + (1-\epsilon) \sum_a \frac{\pi(a|s) - \frac{\epsilon}{|\mathcal{A}(s)|}}{1-\epsilon} Q^\pi(s,a) \\
& && \text{(choose } w(a) \text{ so the } \pi \text{ terms appear)} \\
&= \sum_a \frac{\epsilon}{|\mathcal{A}(s)|} Q^\pi(s,a) - \sum_a \frac{\epsilon}{|\mathcal{A}(s)|} Q^\pi(s,a) + \sum_a \pi(a|s) Q^\pi(s,a) && \text{(rearrange terms)} \\
&= \sum_a \pi(a|s) Q^\pi(s,a) = \underline{V^\pi(s)} && \text{(algebra simplifies to original policy)}
\end{aligned}$$

We just showed: $Q^\pi(s, \pi'(s)) \geq V^\pi(s)$

Therefore: $V_{\pi'}(s) \geq V_\pi(s)$ by the standard Policy Improvement Theorem!

menti.com - Code: **2987 9345**

Why can't on-policy Monte Carlo methods learn the truly optimal policy?

- A) Because they use sample episodes instead of exact expectations
- B) Because they require too many episodes
- C) Because they don't use bootstrapping
- D) Because they must maintain exploration through ϵ-greedy policies
- E) They can learn the truly optimal policy

🎮 Mentimeter Poll: All clear so far?

menti.com — Code: **2987 9345**

Clarity on On-policy Monte Carlo Control?

- A) All clear
- B) Why we use ϵ-greedy unclear
- C) Policy improvement proof unclear
- D) Difference from MC ES unclear

Off-policy

1. Introduction

2. Monte Carlo Policy Evaluation

3. Monte Carlo Control

3.1 On-policy

3.2 Off-policy

Recap: What's a fundamental dilemma in reinforcement learning?

The Exploration-Exploitation Dilemma:

- Want to learn about the **optimal** policy (deterministic, greedy)
- Need to **explore** to find all good actions (stochastic, random)
- Can't do both with the same policy!

On-policy methods compromise: learn near-optimal ϵ-soft policies

What if we could separate these concerns?

How does off-policy learning solve this dilemma?

Off-policy Solution: Use two different policies!

- **Target policy** π: The policy we want to evaluate/improve (can be deterministic)
- **Behavior policy** b: The policy we use to generate data (must be stochastic)
- Learn about π from data generated by b

In order to use episodes from b to estimate values for π, we require the **assumption of coverage**.

Definition (Coverage Assumption)
$\pi(a|s) > 0 \Rightarrow b(a|s) > 0$

Every action taken by π must also be takeable by b

Now we can learn optimal deterministic policies while exploring!

How does off-policy learning solve this dilemma?

Off-policy Solution: Use two different policies!

- **Target policy** π: The policy we want to evaluate/improve (can be deterministic)
- **Behavior policy** b: The policy we use to generate data (must be stochastic)
- Learn about π from data generated by b

In order to use episodes from b to estimate values for π, we require the **assumption of coverage**.

Definition (Coverage Assumption)
$\pi(a|s) > 0 \Rightarrow b(a|s) > 0$

Every action taken by π must also be takeable by b

Now we can learn optimal deterministic policies while exploring!

menti.com - Code: **2987 9345**

The coverage assumption $\pi(a|s) > 0 \Rightarrow b(a|s) > 0$ means:

- A) The behavior policy must be identical to the target policy
- B) Every action must be equally likely under both policies
- C) The behavior policy must try every action the target policy might take
- D) The target policy must explore more than the behavior policy
- E) Both policies must be deterministic

What's the core problem with off-policy learning?

The Problem: Episodes generated by b have wrong probabilities for π

Analogy: Trying to estimate average height of basketball players by sampling from general population - the sample comes from the wrong distribution!

We need: $\mathbb{E}_\pi[G_t|S_t = s] = V^\pi(s)$

We have: $\mathbb{E}_b[G_t|S_t = s] = V^b(s)$

Solution: Importance Sampling!

How does importance sampling work?

Solution: Weight each episode by how likely it would be under π vs b

Importance Sampling Ratio: For trajectory $S_t, A_t, S_{t+1}, A_{t+1}, \ldots, S_T$:

$$\begin{aligned}\rho_{t:T-1} &= \prod_{k=t}^{T-1} \frac{\pi(A_k|S_k)P(S_{k+1}|S_k, A_k)}{b(A_k|S_k)P(S_{k+1}|S_k, A_k)} \\ &= \prod_{k=t}^{T-1} \frac{\pi(A_k|S_k)}{b(A_k|S_k)}\end{aligned}$$

Key Properties:

- Transition probabilities $P(s'|s, a)$ cancel out!
- Only depends on the policies and the sequence
- Transforms expectations: $\mathbb{E}_b[\rho_{t:T-1} G_t | S_t = s] = V^\pi(s)$

How do we average multiple importance-sampled returns?

The Issue: We have multiple visits to state s, each giving us a weighted return ρG_t. How do we combine these?

Ordinary Importance Sampling:

$$V(s) = \frac{\sum_{t \in \mathcal{T}(s)} \rho_{t:T(t)-1} G_t}{|\mathcal{T}(s)|}$$

where, counting time steps across all episodes, $\mathcal{T}(s)$ is the set of time steps at which we visit s, and $T(t)$ is the time the episode containing t terminates (so $\rho_{t:T(t)-1}$ runs from t to the end of that episode).

Weighted Importance Sampling:

$$V(s) = \frac{\sum_{t \in \mathcal{T}(s)} \rho_{t:T(t)-1} G_t}{\sum_{t \in \mathcal{T}(s)} \rho_{t:T(t)-1}}$$

Key difference: Ordinary treats each weighted return *equally*, weighted normalizes by importance weights

Trade-off: Ordinary is unbiased but high variance, weighted is slightly biased but much lower variance

How do we average multiple importance-sampled returns?

The Issue: We have multiple visits to state s, each giving us a weighted return ρG_t. How do we combine these?

Ordinary Importance Sampling:

$$V(s) = \frac{\sum_{t \in \mathcal{T}(s)} \rho_{t:T(t)-1} G_t}{|\mathcal{T}(s)|}$$

where, counting time steps across all episodes, $\mathcal{T}(s)$ is the set of time steps at which we visit s, and $T(t)$ is the time the episode containing t terminates (so $\rho_{t:T(t)-1}$ runs from t to the end of that episode).

Weighted Importance Sampling:

$$V(s) = \frac{\sum_{t \in \mathcal{T}(s)} \rho_{t:T(t)-1} G_t}{\sum_{t \in \mathcal{T}(s)} \rho_{t:T(t)-1}}$$

Key difference: Ordinary treats each weighted return *equally*, weighted normalizes by importance weights

Trade-off: Ordinary is unbiased but high variance, weighted is slightly biased but much lower variance

How do we average multiple importance-sampled returns?

The Issue: We have multiple visits to state s, each giving us a weighted return ρG_t. How do we combine these?

Ordinary Importance Sampling:

$$V(s) = \frac{\sum_{t \in \mathcal{T}(s)} \rho_{t:T(t)-1} G_t}{|\mathcal{T}(s)|}$$

where, counting time steps across all episodes, $\mathcal{T}(s)$ is the set of time steps at which we visit s, and $T(t)$ is the time the episode containing t terminates (so $\rho_{t:T(t)-1}$ runs from t to the end of that episode).

Weighted Importance Sampling:

$$V(s) = \frac{\sum_{t \in \mathcal{T}(s)} \rho_{t:T(t)-1} G_t}{\sum_{t \in \mathcal{T}(s)} \rho_{t:T(t)-1}}$$

Key difference: Ordinary treats each weighted return *equally*, weighted normalizes by importance weights

Trade-off: Ordinary is unbiased but high variance, weighted is slightly biased but much lower variance

How do we average multiple importance-sampled returns?

The Issue: We have multiple visits to state s, each giving us a weighted return ρG_t. How do we combine these?

Ordinary Importance Sampling:

$$V(s) = \frac{\sum_{t \in \mathcal{T}(s)} \rho_{t:T(t)-1} G_t}{|\mathcal{T}(s)|}$$

where, counting time steps across all episodes, $\mathcal{T}(s)$ is the set of time steps at which we visit s, and $T(t)$ is the time the episode containing t terminates (so $\rho_{t:T(t)-1}$ runs from t to the end of that episode).

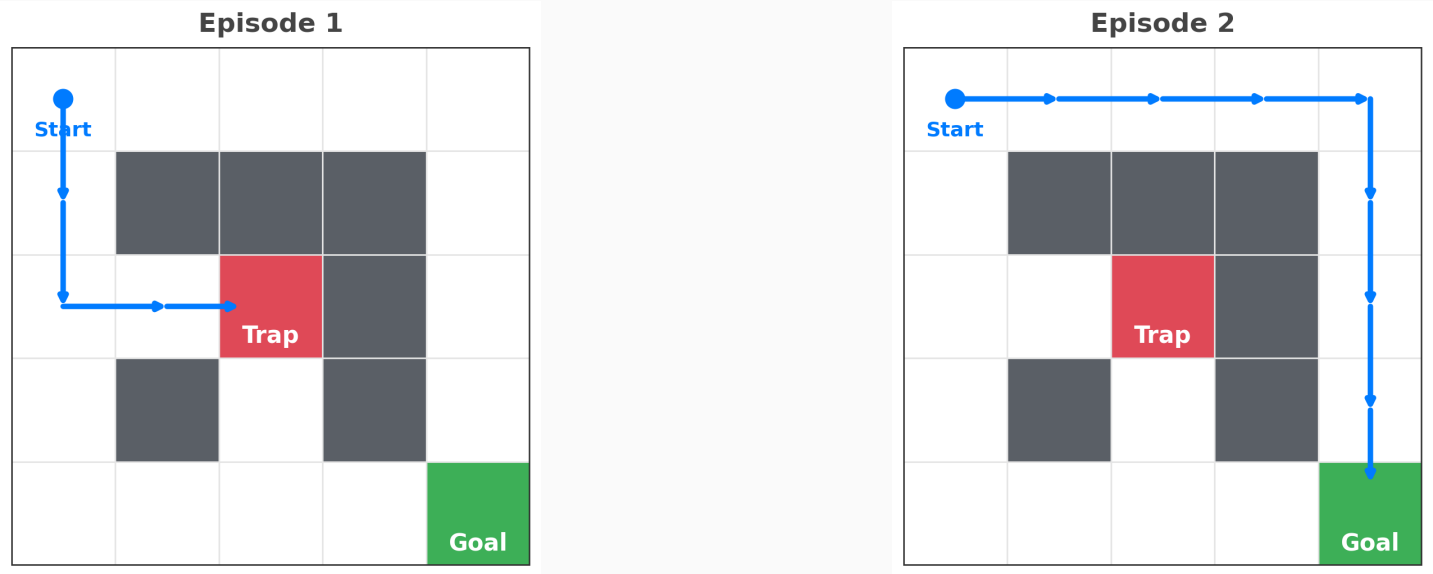
Weighted Importance Sampling:

$$V(s) = \frac{\sum_{t \in \mathcal{T}(s)} \rho_{t:T(t)-1} G_t}{\sum_{t \in \mathcal{T}(s)} \rho_{t:T(t)-1}}$$

Key difference: Ordinary treats each weighted return *equally*, weighted normalizes by importance weights

Trade-off: Ordinary is unbiased but high variance, weighted is slightly biased but much lower variance

Example 1: Off-policy Monte Carlo on GridWorld

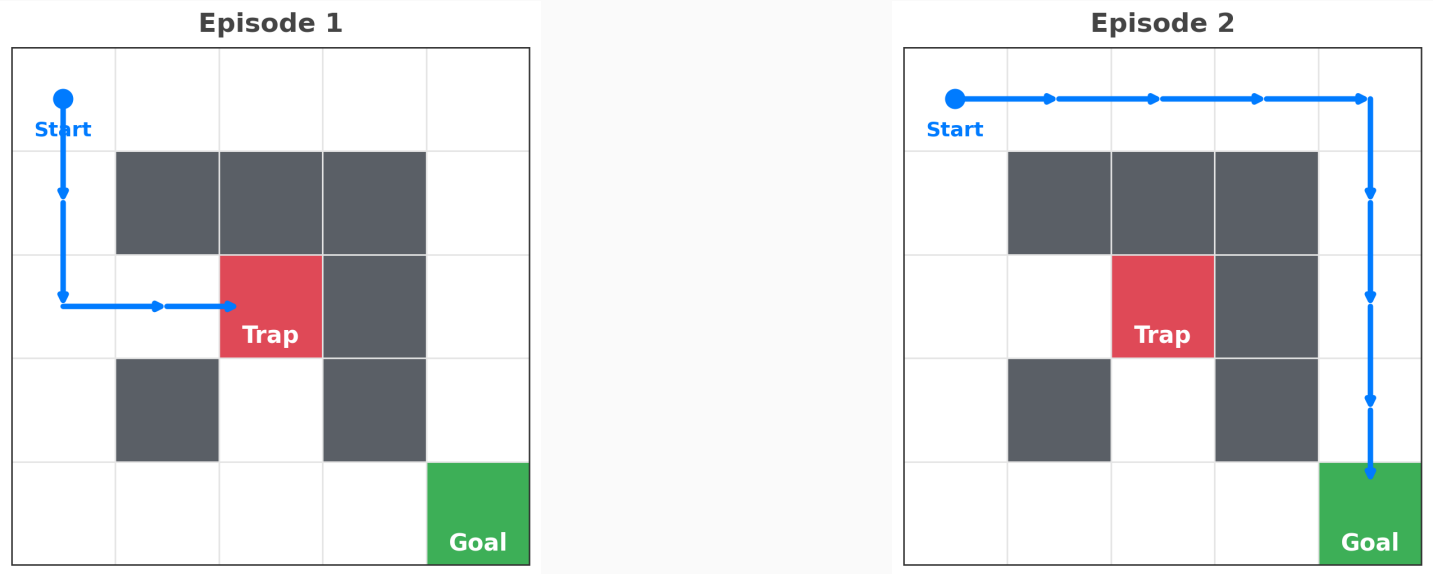


Our two episodes were generated by a behaviour policy b: uniform random, $b(a|s) = 0.25$.

We want V^π **for a target policy** π that prefers Right/Down: $\pi = 0.4$ for each, 0.1 for Up/Left.

Both episodes only use Right/Down, so every step has ratio $\frac{\pi}{b} = \frac{0.4}{0.25} = 1.6$ and $\rho_{t:T-1} = 1.6^{T-t}$.

Example 1: Off-policy Monte Carlo on GridWorld

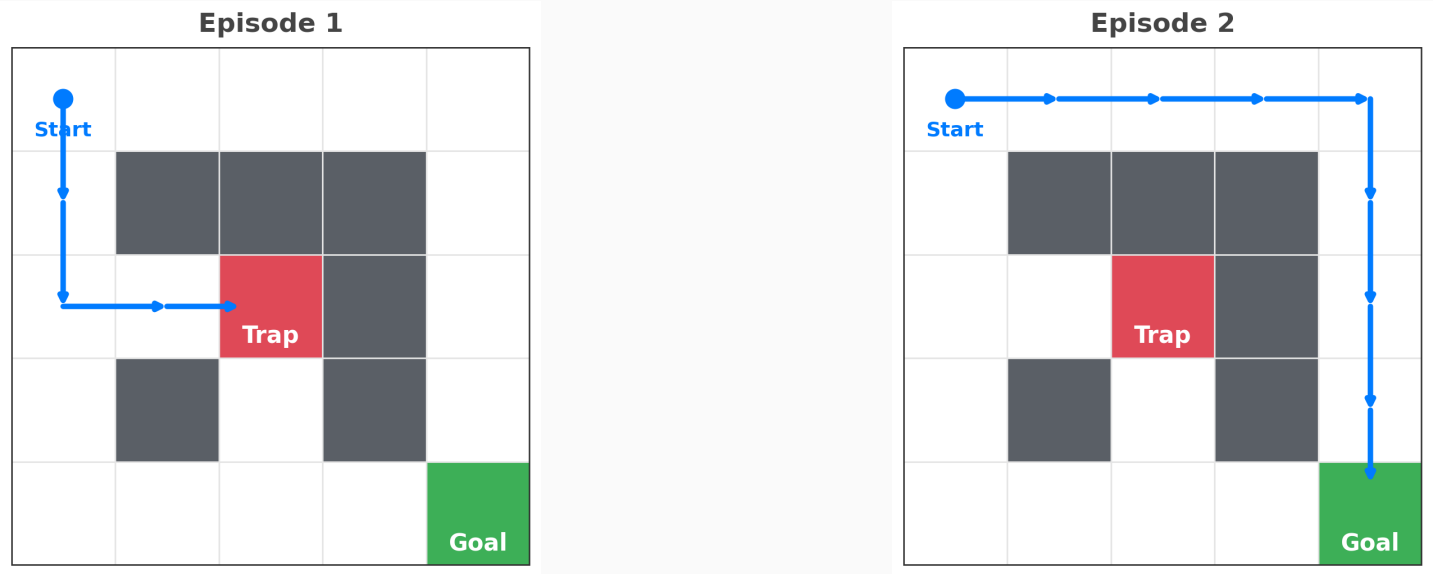


Our two episodes were generated by a behaviour policy b: uniform random, $b(a|s) = 0.25$.

We want V^π **for a target policy** π that prefers Right/Down: $\pi = 0.4$ for each, 0.1 for Up/Left.

Both episodes only use Right/Down, so every step has ratio $\frac{\pi}{b} = \frac{0.4}{0.25} = 1.6$ and $\rho_{t:T-1} = 1.6^{T-t}$.

Example 1: Off-policy Monte Carlo on GridWorld

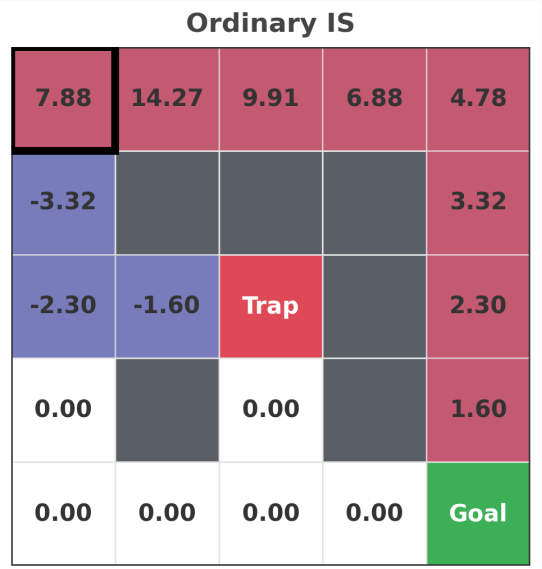


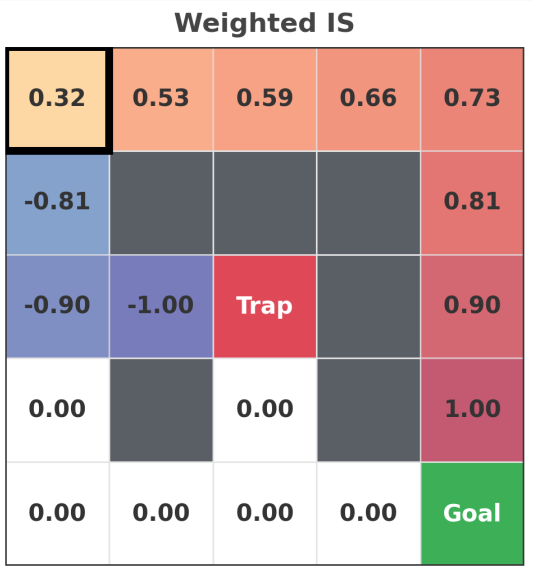
Our two episodes were generated by a behaviour policy b: uniform random, $b(a|s) = 0.25$.

We want V^π **for a target policy** π that prefers Right/Down: $\pi = 0.4$ for each, 0.1 for Up/Left.

Both episodes only use Right/Down, so every step has ratio $\frac{\pi}{b} = \frac{0.4}{0.25} = 1.6$ and $\rho_{t:T-1} = 1.6^{T-t}$.

Example 1: Ordinary vs weighted importance sampling after two rollouts



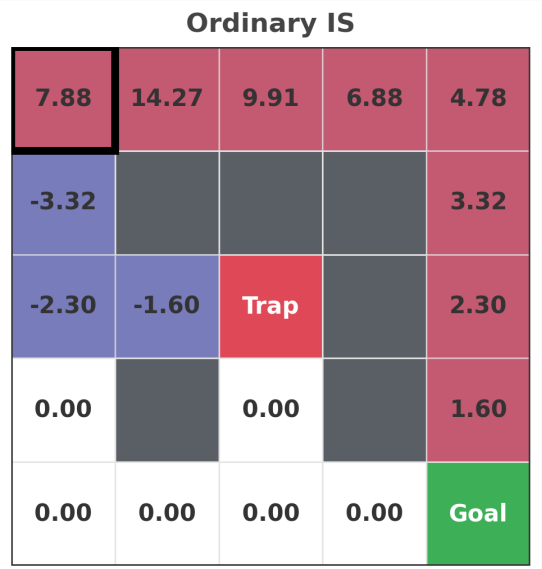


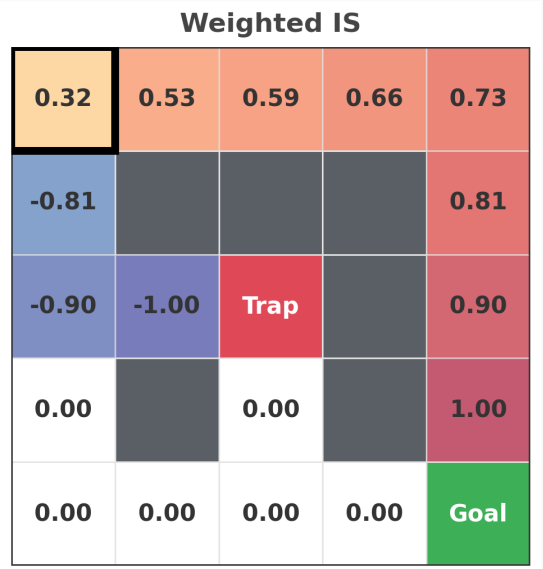
Breaking down cell $(0, 0)$:

	$\rho_{0:T-1}$	Return G_0	$\rho_{0:T-1}\, G_0$
Episode 1 (trap, $T = 4$)	$1.6^4 = 6.55$	-0.729	-4.78
Episode 2 (goal, $T = 8$)	$1.6^8 = 42.95$	0.478	20.54

Ordinary: $\frac{-4.78+20.54}{2} = 7.88$ Weighted: $\frac{-4.78+20.54}{6.55+42.95} = 0.32$

Example 1: Ordinary vs weighted importance sampling after two rollouts



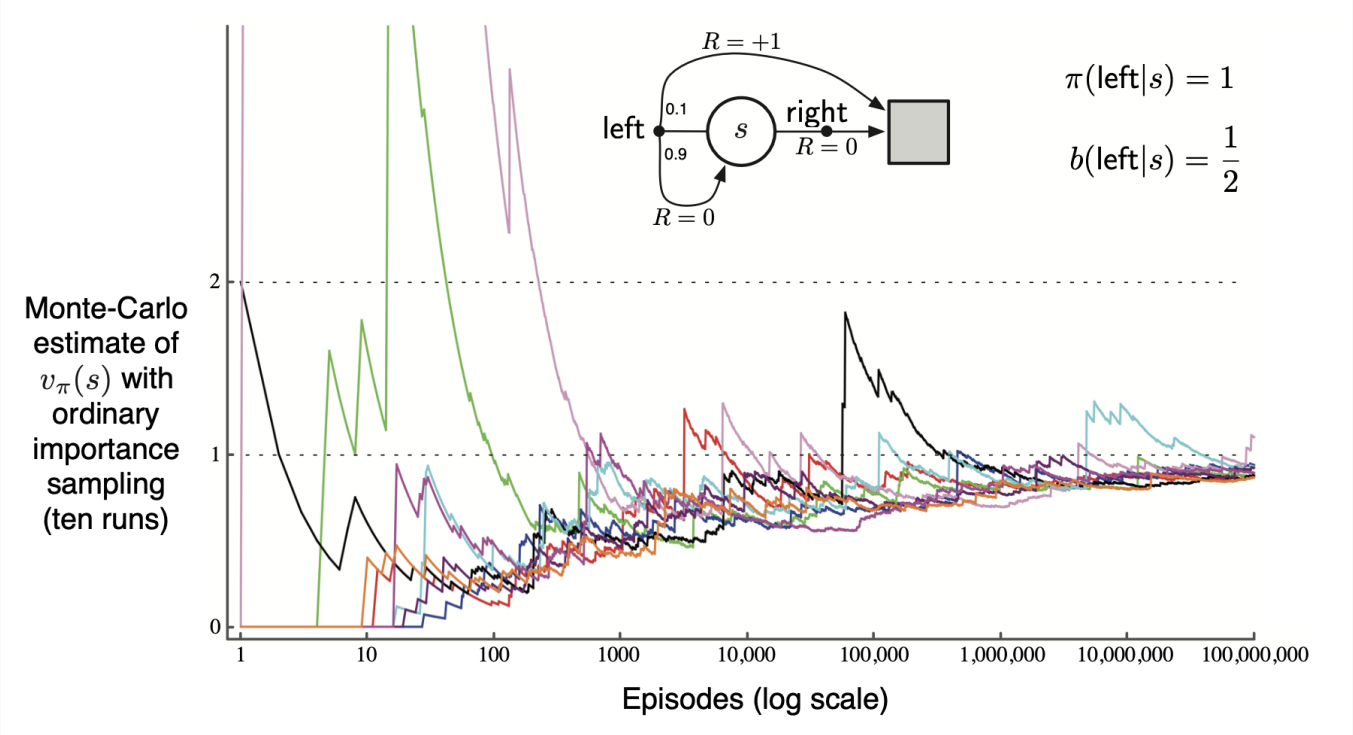


Breaking down cell $(0, 0)$:

	$\rho_{0:T-1}$	Return G_0	$\rho_{0:T-1}\, G_0$
Episode 1 (trap, $T = 4$)	$1.6^4 = 6.55$	-0.729	-4.78
Episode 2 (goal, $T = 8$)	$1.6^8 = 42.95$	0.478	20.54

Ordinary: $\frac{-4.78+20.54}{2} = 7.88$ Weighted: $\frac{-4.78+20.54}{6.55+42.95} = 0.32$

Example 2: Why can ordinary importance sampling fail catastrophically?



Ordinary importance sampling fails to converge on simple MDP

True value: $V^\pi(s) = 1$ (when $\gamma = 1$)

Result: Even after 100 million episodes, estimates don't converge!

Why: Because $\frac{\pi(a|s)}{b(a|s)} = 2$, episode with k actions have weight $2^k \rightarrow$ extremely high variance!

🎮 Mentimeter Poll

menti.com - Code: **2987 9345**

What's the main reason weighted importance sampling is preferred over ordinary importance sampling?

- A) It's unbiased while ordinary is biased
- B) It's easier to implement
- C) It has much lower variance
- D) It doesn't require importance sampling ratios

How do we implement Off-policy MC Prediction efficiently?

The storage problem exists for ALL Monte Carlo methods:

- Memory grows without bound: Returns$(s, a) = [G_1, G_2, G_3, \ldots]$
- This applied to on-policy MC control too!

But off-policy methods have additional storage needs:

- Store importance sampling ratios: $W_1, W_2, W_3, \ldots$
- Store cumulative weights: $C_1, C_2, C_3, \ldots$
- More complex averaging: $Q(s, a) = \frac{\sum_i W_i G_i}{\sum_i W_i}$

We need incremental updates for practical implementation!

Incremental updates solve both problems

For on-policy methods (simple average):

$$Q_{n+1} = Q_n + \frac{1}{n}[G_n - Q_n]$$

where n is the number of returns observed so far.

For off-policy methods (weighted average):

$$Q_{n+1} = Q_n + \frac{W_n}{C_n}[G_n - Q_n] \tag{2}$$

$$C_{n+1} = C_n + W_n \tag{3}$$

where C_n is cumulative sum of importance weights, W_n is current importance ratio.

Off-policy MC Prediction Algorithm

Off-policy MC Prediction for estimating $Q \approx Q^{\pi}$

Input: Target policy π to be evaluated
Initialize: For all $s \in \mathcal{S}, a \in \mathcal{A}(s)$:
$Q(s,a) \leftarrow$ arbitrary, $C(s,a) \leftarrow 0$

repeat
  $b \leftarrow$ any policy with coverage of π
  Generate episode using b: $S_0, A_0, R_1, \ldots, S_{T-1}, A_{T-1}, R_T$
  $G \leftarrow 0$, $W \leftarrow 1$
  for $t = T-1, T-2, \ldots, 0$ while $W \neq 0$ **do**
    $G \leftarrow \gamma G + R_{t+1}$
    $C(S_t, A_t) \leftarrow C(S_t, A_t) + W$
    $Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \frac{W}{C(S_t, A_t)}[G - Q(S_t, A_t)]$
    $W \leftarrow W \cdot \frac{\pi(A_t|S_t)}{b(A_t|S_t)}$
  end for
until converged

From off-policy prediction to off-policy control

What we've accomplished so far:

- Off-policy prediction: Estimate Q^π for a fixed target policy π
- Used importance sampling to correct for behavior policy bias
- Incremental updates for practical implementation

But for control, we want to find the optimal policy:

- Don't just evaluate a given policy - improve it!
- Apply the GPI pattern: evaluation + improvement

The key idea:

- Target policy: $\pi(s) = \text{argmax}_a Q(s, a)$ (deterministic, greedy)
- Behavior policy: b remains stochastic for exploration (e.g., ϵ-greedy)

Result: Learn the optimal deterministic policy while exploring with a stochastic policy!

From off-policy prediction to off-policy control

What we've accomplished so far:

- Off-policy prediction: Estimate Q^π for a fixed target policy π
- Used importance sampling to correct for behavior policy bias
- Incremental updates for practical implementation

But for control, we want to find the optimal policy:

- Don't just evaluate a given policy - improve it!
- Apply the GPI pattern: evaluation + improvement

The key idea:

- Target policy: $\pi(s) = \text{argmax}_a Q(s, a)$ (deterministic, greedy)
- Behavior policy: b remains stochastic for exploration (e.g., ϵ-greedy)

Result: Learn the optimal deterministic policy while exploring with a stochastic policy!

From off-policy prediction to off-policy control

What we've accomplished so far:

- Off-policy prediction: Estimate Q^π for a fixed target policy π
- Used importance sampling to correct for behavior policy bias
- Incremental updates for practical implementation

But for control, we want to find the optimal policy:

- Don't just evaluate a given policy - improve it!
- Apply the GPI pattern: evaluation + improvement

The key idea:

- Target policy: $\pi(s) = \text{argmax}_a Q(s, a)$ (deterministic, greedy)
- Behavior policy: b remains stochastic for exploration (e.g., ϵ-greedy)

Result: Learn the optimal deterministic policy while exploring with a stochastic policy!

From off-policy prediction to off-policy control

What we've accomplished so far:

- Off-policy prediction: Estimate Q^π for a fixed target policy π
- Used importance sampling to correct for behavior policy bias
- Incremental updates for practical implementation

But for control, we want to find the optimal policy:

- Don't just evaluate a given policy - improve it!
- Apply the GPI pattern: evaluation + improvement

The key idea:

- Target policy: $\pi(s) = \text{argmax}_a Q(s, a)$ (deterministic, greedy)
- Behavior policy: b remains stochastic for exploration (e.g., ϵ-greedy)

Result: Learn the optimal deterministic policy while exploring with a stochastic policy!

Off-policy Monte Carlo Control

Off-policy MC Control Algorithm

Initialize:

$Q(s,a) \leftarrow$ arbitrary, $C(s,a) \leftarrow 0$, $\pi(s) \leftarrow \text{argmax}_a Q(s,a)$

repeat

  $b \leftarrow$ any soft policy

  Generate episode using b: $S_0, A_0, R_1, \ldots, S_{T-1}, A_{T-1}, R_T$

  $G \leftarrow 0$, $W \leftarrow 1$

  for $t = T-1, T-2, \ldots, 0$ **do**

    $G \leftarrow \gamma G + R_{t+1}$

    $C(S_t, A_t) \leftarrow C(S_t, A_t) + W$

    $Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \frac{W}{C(S_t, A_t)}[G - Q(S_t, A_t)]$

    $\pi(S_t) \leftarrow \text{argmax}_a Q(S_t, a)$

    If $A_t \neq \pi(S_t)$ **then** exit inner loop

    $W \leftarrow W \cdot \frac{1}{b(A_t|S_t)}$

  end for

until converged

Why $\frac{1}{b(A_t|S_t)}$? Because $\pi(a|s)$ is deterministic, so $\frac{\pi(a|s)}{b(a|s)}$ is either $\frac{1}{b(a|s)}$ if $a = \pi(s)$ or 0 otherwise.

Off-policy MC Control Algorithm

Initialize:

$Q(s,a) \leftarrow$ arbitrary, $C(s,a) \leftarrow 0$, $\pi(s) \leftarrow \text{argmax}_a Q(s,a)$

repeat

  $b \leftarrow$ any soft policy

  Generate episode using b: $S_0, A_0, R_1, \ldots, S_{T-1}, A_{T-1}, R_T$

  $G \leftarrow 0$, $W \leftarrow 1$

  for $t = T-1, T-2, \ldots, 0$ **do**

    $G \leftarrow \gamma G + R_{t+1}$

    $C(S_t, A_t) \leftarrow C(S_t, A_t) + W$

    $Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \frac{W}{C(S_t, A_t)}[G - Q(S_t, A_t)]$

    $\pi(S_t) \leftarrow \text{argmax}_a Q(S_t, a)$

    If $A_t \neq \pi(S_t)$ **then** exit inner loop

    $W \leftarrow W \cdot \frac{1}{b(A_t|S_t)}$

  end for

until converged

Why $\frac{1}{b(A_t|S_t)}$**?** Because $\pi(a|s)$ is deterministic, so $\frac{\pi(a|s)}{b(a|s)}$ is either $\frac{1}{b(a|s)}$ if $a = \pi(s)$ or 0 otherwise.

🎮 Mentimeter Poll: All clear so far?

menti.com — Code: **2987 9345**

Clarity on Off-policy Monte Carlo?

- A) All clear
- B) Why we need importance sampling unclear
- C) How importance ratios work unclear
- D) Difference between ordinary and weighted IS unclear
- E) Why off-policy can learn optimal policies unclear

What are the main limitations of Monte Carlo methods?

Episode-based learning:

- Only works for episodic tasks
- Must wait for episode completion to update
- No online learning during episodes
- Can be slow if episodes are long

High variance:

- Sample returns can vary greatly
- Especially problematic in off-policy methods
- May need many episodes for good estimates
- Importance sampling can make variance worse

What other limitations should we be aware of?

No bootstrapping:

- Can't use knowledge of value function structure
- Slower convergence than methods that bootstrap
- Each state estimate is independent

Exploration challenges:

- Off-policy methods can be data inefficient
- Importance ratios can have high variance
- Coverage assumption can be restrictive

Next week: Temporal Difference learning addresses these limitations!

menti.com - Code: **2987 9345**

What is the main advantage of off-policy over on-policy methods?

- A) Off-policy methods converge faster
- B) Off-policy methods are easier to implement
- C) Off-policy can learn optimal deterministic policies while exploring
- D) Off-policy methods require less memory
- E) Off-policy methods work better with function approximation

menti.com - Code: **2987 9345**

What is the biggest limitation of basic Monte Carlo methods?

- A) They require a model of the environment
- B) They cannot find optimal policies
- C) They cannot handle stochastic policies
- D) They are computationally too expensive
- E) They only work for episodic tasks

What have we learned today?

Key Takeaways

- **Model-free learning**: MC learns directly from experience without environment model
- **Sample-based estimation**: Use Law of Large Numbers to estimate value functions
- **Exploration strategies**: Exploring starts or ϵ-soft policies ensure adequate exploration
- **Monte Carlo control**: Apply GPI with MC policy evaluation for optimal policies
- **Off-policy learning**: Use importance sampling to learn about one policy while following another
- **Variance trade-offs**: Weighted importance sampling preferred over ordinary in practice

Next Week: Temporal Difference Learning

- Combine sampling (like MC) with bootstrapping (like DP)
- Enable online learning and handle continuing tasks
- Foundation for Q-learning and modern deep RL

Reading and Resources

📚 Essential Reading

- **Sutton & Barto**: Chapter 5 (Monte Carlo Methods)
- Focus on: MC Prediction (5.1), MC Estimation of Action Values (5.2), MC Control (5.3)
- Important: On-policy vs Off-policy (5.4), Importance Sampling (5.5)
- Advanced: Incremental Implementation (5.6), Off-policy Control (5.7)

🎮 Lecture Feedback

menti.com - Code: **2987 9345**

Q1: If you got lost today, at what point did you lose track?

- A) MC vs DP differences
- B) MC Policy Evaluation
- C) MC Control and exploring starts
- D) On-policy methods and ϵ-greedy
- E) Off-policy methods and importance sampling
- F) Ordinary vs weighted importance sampling
- G) Off-policy MC Control algorithm
- H) None of the above, I followed everything

Q2: Generic feedback - what worked well, what could be improved?

- e.g. “Pace was too fast/slow in the off-policy section.”